Explainable Fraud Detection and Tamper-Evident Transaction Monitoring for On-Chain Finance

Rashmi DT
School of Computer Science and Engineering
RV University
Bengaluru, India
rashmideepaktoragallamathbtech24@rvu.edu.in

Bhoomika P
School of Computer Science and Engineering
RV University
Bengaluru, India
bhoomikapbtech24@rvu.edu.in

Anita
School of Computer Science and Engineering
RV University
Bengaluru, India
anitabtech24@rvu.edu.in

Dr.Cynthia J
School of Computer Science and Engineering
RV University
Bengaluru, India
cynthiaj@rvu.edu.in

***Abstract*—Public blockchains let anyone move funds through new pseudonymous wallets, yet no central party screens those transfers for money-laundering behaviour, and the alert records that compliance teams keep can be edited after the fact. This paper presents a prototype monitoring layer that addresses both gaps. A Python engine reads live Ethereum mainnet blocks through a read-only JSON-RPC connection, applies eight transparent rules for known laundering typologies together with an Isolation Forest outlier detector, and merges the signals with a noisy-OR formula into a single 0–100 risk score whose reasons are named. Analysts review one case per wallet in a dashboard whose workflow mirrors a smart contract. Approved alerts are anchored in `RiskRegistry`, a Solidity contract on the Sepolia testnet that stores only a transaction hash, a wallet, a score, a reason bitmask and a review status, protected by role-based access, duplicate detection and final review states. On a simulated data set with 42 injected fraud scenarios, all 42 were detected; alert precision was 0.996 with an exchange allowlist and 0.606 without it. The contract passes 15 test cases, eight of them negative, and real alerts were anchored on Sepolia for 585,087 to 683,007 gas each. The detection results come from simulated fraud and show that the pipeline works as designed; they are not an estimate of accuracy on real laundering.**

***Index Terms*—blockchain, Ethereum, anti-money laundering, fraud detection, Isolation Forest, smart contract, audit trail**

I. Introduction

Ethereum allows anyone to create a wallet without identity checks and to move value within seconds [7]. That openness is valuable for finance, but it also makes it easy to hide the origin of funds by splitting, relaying and returning them through many fresh wallets. Regulators extend anti-money-laundering (AML) duties to businesses that handle virtual assets and ask them to apply a risk-based approach and to monitor transactions [6]. The chain is public, so the raw data needed for such monitoring already exists. What is missing is a tool that turns that data into alerts an analyst can understand, defend, and later prove were not altered.

Three practical problems follow. First, manual review of public chain data is slow, and patterns such as one wallet fanning funds out to dozens of recipients are easy to miss by eye. Second, opaque scores are hard to justify to an auditor, while rule-only engines are easy to evade because their thresholds are guessable. Third, alert logs and review decisions normally live in an ordinary database, so nothing shows that an alert was not deleted or a decision changed afterwards.

This paper makes three contributions:

1) an explainable scoring method that combines transparent AML rules with an unsupervised outlier detector so that every score decomposes exactly into named reasons;
2) `RiskRegistry`, a role-based smart contract that records alerts and review decisions as permanent events, together with an off-chain review workflow that obeys the same state rules; and
3) a working prototype with a live mainnet mode, an analyst dashboard, an automated test suite, a security review and a measured on-chain cost.

Section II gives background and related work, Section III describes the detection pipeline, Section IV the smart contract, Section V the security analysis, Section VI the implementation, Section VII the design rationale, Section VIII the evaluation, and Sections IX and X discuss limitations and conclude.

II. Background and Related Work

A. Laundering patterns on a public ledger

AML practice describes laundering as placement, layering and integration. On a public chain the layering stage leaves recognisable footprints. *Structuring* (or smurfing) splits an amount into several transfers that each stay just under a review limit. *Fan-out* spreads funds from one wallet to many recipients in a short time, and *fan-in* gathers funds from many wallets into one. *Round-tripping* sends funds away and brings them back to the origin. Other signals are very high transfer velocity, unusually large values, freshly created wallets that immediately move large amounts, and contact with wallets on a watchlist. None of these signals proves wrongdoing; exchanges and market makers, for example, legitimately show high velocity and fan-out. A practical system must therefore explain each alert and keep a human in the loop.

TABLE I
COMPARISON WITH RELATED WORK

Work	Approach	Gap addressed here
[1]	GCN on labelled Bitcoin graph	Offline; no alert workflow or audit trail
[2]	Graph features + LightGBM	Supervised; account labels needed; no explained, recorded alerts
[3]	Survey of anomaly detection	No working monitoring layer
[4]	Taxonomy of DeFi fraud methods	Review only; early detection immature
[5]	Isolation Forest	Needs AML features, rules and an explanation layer
This work	Rules + Isolation Forest + on-chain audit trail	Explained live alerts with tamper-evident review

B. Related work

Weber *et al.* [1] applied graph convolutional networks to a labelled Bitcoin transaction graph and showed that graph structure helps identify illicit transactions, but their study is an offline experiment on a single labelled data set. Chen *et al.* [2] detected phishing accounts on Ethereum by extracting graph-based cascade features and training a LightGBM ensemble; the method needs labelled phishing accounts and classifies accounts rather than producing explained alerts with a recorded review. Hassan *et al.* [3] survey anomaly detection in blockchain networks, covering evaluation metrics, requirements and models by blockchain layer, and list open challenges. Luo *et al.* [4] organise DeFi fraud by project life-cycle stage and review statistical, machine-learning and graph methods, noting that early-stage detection is immature because data and reliable ground truth are scarce. Liu *et al.* [5] introduced Isolation Forest, which isolates outliers with random trees and needs no labelled fraud, a property that suits a setting where labels are rare.

The works in Table I either detect well offline or survey the field. None combines real-time monitoring, scores that decompose into readable reasons, and a tamper-evident record of the analyst's decision. Our work targets that combination.

III. DETECTION PIPELINE

A. Architecture

Detection runs off-chain, because scanning every transfer on-chain would be prohibitively expensive, and the chain is used only for a permanent record. Fig. 1 shows the data flow. Blocks are read from an Ethereum node over read-only JSON-RPC and native ETH transfers are stored in SQLite. Features, rules and the outlier detector produce a score with reason codes. Alerts are grouped into one case per wallet. An analyst reviews a case and may queue it for anchoring. A separate script signs and sends the transaction to `RiskRegistry`, so the web server never holds a private key.

TABLE II
RULES, CONDITIONS AND WEIGHTS

Reason	Condition (1-hour window)	w
Structuring	≥ 3 transfers of 9–10 ETH, just under a 10 ETH limit	0.55
Fan-out	one sender, ≥ 15 distinct recipients	0.50
Fan-in	≥ 15 senders into one receiver	0.50
Round-trip	funds return to origin within $\pm 25\%$	0.50
High velocity	≥ 20 transfers by one sender	0.50
Watchlist	counterparty on the watchlist file	0.75
Large value	transfer of ≥ 100 ETH	0.35
New-wallet burst	nonce ≤ 5 and ≥ 20 ETH moved	0.35
ML outlier	Isolation Forest, top 1% most isolated	0.30

B. Ingestion

The ingestion module calls standard Ethereum JSON-RPC methods and stores, for each native ETH transfer, the hash, block, timestamp, sender, receiver, value and sender nonce. A cursor lets a restart continue where it stopped. The client uses https only, redacts the API key from logs, retries with back-off on network errors, accepts compressed responses, and tolerates truncated reads. For repeatable experiments a simulator generates a realistic background of normal transfers and injects labelled fraud scenarios.

C. Features and rules

For each transfer the engine computes behavioural features over a one-hour sliding window. Table II lists the eight rules, the condition each uses in the prototype, and its weight. The conditions map to the typologies of Section II-A. Thresholds are configurable; they were chosen by the team for the prototype and are not calibrated against real compliance data.

D. Isolation Forest

Rules capture known patterns; an unsupervised detector can flag behaviour that no rule describes. We train an Isolation Forest [5] from scikit-learn [8] on the behavioural feature vectors. A transfer whose anomaly score falls in the top 1% is marked as an ML outlier, but only once at least 200 samples are available, so that a nearly empty database does not produce spurious outliers. Because "unusual" is not the same as "wrong", the outlier signal has the lowest weight and cannot raise an alert alone.

E. Explainable risk score

Each signal i carries a weight $w_i \in (0, 1)$, read as how strongly the signal alone suggests risk. Independent signals combine with a noisy-OR rule:

$$S \;=\; 100\Big(1 - \prod_{i \in \mathcal{H}} (1 - w_i)\Big), \tag{1}$$

where $\mathcal{H}$ is the set of signals that fired. The score stays below 100, never decreases when evidence is added, and decomposes exactly into the reasons that produced it. Pattern

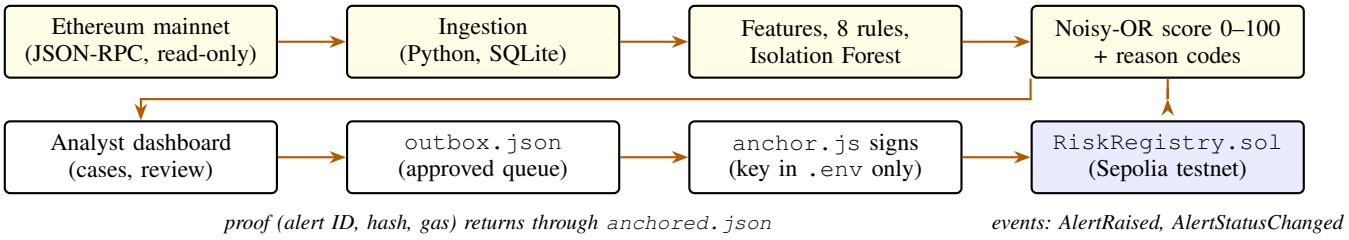


Fig. 1. Architecture. Shaded boxes: off-chain detection (yellow) and the on-chain contract (blue). Only the anchoring script holds a key.

Algorithm 1 Scoring one transfer

$\mathcal{H} \leftarrow \emptyset$; $mask \leftarrow 0$
for all rule r in the eight rules **do**
  if r fires on the window features **then**
    $\mathcal{H} \leftarrow \mathcal{H} \cup \{r\}$; $mask \leftarrow mask \mid (1 \ll bit(r))$
  end if
end for
if Isolation Forest flags the feature vector **then**
  add ML outlier to $\mathcal{H}$ and $mask$
end if
$S \leftarrow \lfloor 100(1 - \prod_{i \in \mathcal{H}}(1 - w_i)) \rfloor$
if $S \geq 50$ **then**
  raise alert $(hash, subject, S, mask)$ with named reasons
end if

rules (weights 0.50–0.55) and the watchlist rule (0.75) can raise an alert alone; context rules (large value, new wallet, outlier; 0.30–0.35) stay below the alert threshold of 50 and need corroboration. Severity is *critical* for $S \geq 80$, *high* for $65 \leq S < 80$ and *medium* for $50 \leq S < 65$. For example, high velocity (0.50), fan-out (0.50) and outlier (0.30) give $100(1 - 0.5 \cdot 0.5 \cdot 0.7) = 82.5$, reported as 82. Algorithm 1 summarises scoring.

F. Cases and review workflow

One wallet can raise many alerts, so the dashboard groups alerts into one *case* per wallet; for fan-in alerts the case belongs to the receiver. An analyst can mark a case Reviewed, Escalated or Dismissed. The allowed moves are exactly those of the contract (Fig. 2): Open may move to any other state, Reviewed may move to Escalated or Dismissed, and Escalated and Dismissed are final. A known exchange can be allowlisted, which dismisses its case and suppresses later behaviour-based alerts for that wallet. Every signal carries a plain-language explanation, why it matters, and a suggested next step, shown next to the evidence so the analyst can justify the decision.

G. Dashboard and API

The dashboard is served by a small standard-library HTTP server bound to localhost. Its read-only endpoints feed the page, and three write endpoints change a case (Table III). The case panel is organised as *Detect* (score, severity, signals), *Explain* (why each signal matters and the next step, with the highest-scoring evidence and recent transfers) and *Prove* (the on-chain alert ID, transaction link and gas once anchored). Wallets whose behaviour resembles an exchange or bot carry a visible label, because high-volume services are the main source of false positives.

TABLE III
LOCAL API USED BY THE DASHBOARD

Endpoint	Purpose
GET `/api/summary`	Tiles, charts, signal counts
GET `/api/cases`	Case list; filters `status`, `min_score`, `q`
GET `/api/cases/{w}`	Case detail, explanations, evidence
POST `.../status`	Reviewed, escalated or dismissed, with note
POST `.../queue`	Queue or unqueue for anchoring
POST `.../allowlist`	Allowlist a wallet and dismiss its case

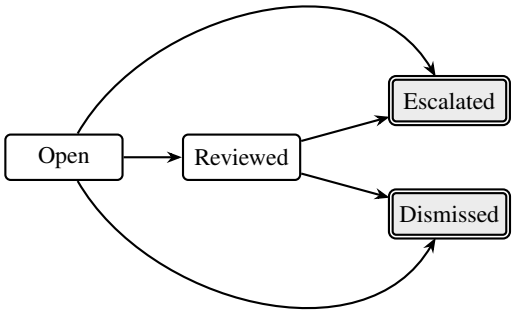


Fig. 2. Review states shared by the dashboard and the contract. Double borders mark final states.

IV. Smart Contract Design

`RiskRegistry` (Solidity 0.8.24) is the on-chain audit trail. It stores no personal data: only a transaction hash, a public wallet address, a score from 1 to 100 and a 16-bit reason mask.

A. Data model and access control

Each alert is a struct holding the monitored transaction hash, the flagged wallet, the analyst who anchored it, two timestamps, the reason mask, the score and the status (an

Listing 1. `raiseAlert` from `RiskRegistry.sol` (event emission shortened).

```
function raiseAlert(bytes32 txHash, address subject,
    uint8 riskScore, uint16 reasonMask)
    external onlyAnalyst returns (uint256 alertId)
{
  if (txHash == bytes32(0)) revert EmptyTxHash();
  if (subject == address(0)) revert ZeroAddress();
  if (riskScore == 0 || riskScore > MAX_RISK_SCORE)
      revert InvalidRiskScore(riskScore);
  if (reasonMask == 0) revert NoReasonGiven();

  uint256 existing = _alertIdPlusOne[txHash];
  if (existing != 0)
      revert AlreadyReported(txHash, existing - 1);

  alertId = _alerts.length;
  _alerts.push(Alert({ txHash: txHash, subject: subject,
      raisedBy: msg.sender,
      raisedAt: uint64(block.timestamp),
      updatedAt: uint64(block.timestamp),
      reasonMask: reasonMask, riskScore: riskScore,
      status: Status.Open }));
  _alertIdPlusOne[txHash] = alertId + 1;
  emit AlertRaised(alertId, txHash, subject,
                   riskScore, reasonMask);
}
```

TABLE IV
REASON BIT POSITIONS (SHARED BY ENGINE AND CONTRACT)

Bit	Reason	Bit	Reason	Bit	Reason
0	Large value	3	Fan-out	6	Watchlist
1	High velocity	4	Fan-in	7	New-wallet burst
2	Structuring	5	Round-trip	8	ML outlier

enum with the four review states). Alerts live in an array; a mapping from transaction hash to alert index lets the contract reject a second report of the same transaction. The deployer is the owner and the first analyst. Only analysts may write alerts or change status. Only the owner manages analysts and a watchlist of flagged addresses, and ownership transfers in two steps (propose, then accept) so that a mistyped address cannot lock the contract.

B. Functions and events

`raiseAlert` validates its inputs and stores the alert; `updateStatus` applies the transition rules of Fig. 2. Listing 1 shows the core of `raiseAlert`. The contract uses custom errors rather than revert strings to save gas and to give precise failure causes, and emits an event for every action (`AlertRaised`, `AlertStatusChanged`, `WatchlistUpdated`, `AnalystUpdated`, ownership events). Events make the audit trail readable by any explorer or indexer without paying for extra storage reads.

C. Reason bitmask

Each reason owns one bit, and the Python engine and the contract use the same positions (Table IV). A mask therefore stores a whole set of reasons in two bytes, and a viewer can decode it without the engine. For instance, mask 266 equals $2^8 + 2^3 + 2^1$ and means ML outlier, fan-out and high velocity.

TABLE V
THREATS AND MITIGATIONS

Threat	Mitigation
Key theft	Secrets only in a git-ignored `.env`; throwaway testnet wallet; only the anchor script signs; mainnet access is read-only
Unauthorised writes	`onlyAnalyst` / `onlyOwner`; two-step ownership transfer
Replay or double reporting	One alert per transaction hash; duplicate reverts
Tampering with history	Alerts are append-only; final states cannot change; every change emits an event
Invalid data on-chain	Score range, non-empty reasons, non-zero hash and wallet enforced
Cross-site requests	Localhost bind; Origin check; custom header on writes
DNS rebinding	Host header check
Script injection	Strict content-security policy; data rendered as text only
Oversized requests	Body size limits; bounded draining of refused bodies
Secret leakage in logs	API key redacted from errors; https only
Bad queue items	Anchor script validates each item, skips invalid ones, never sends a transition the contract would revert, caps 10 transactions per run

V. Security Analysis

Table V lists the main threats considered and the measure that addresses each.

VI. Implementation

The engine is written in Python 3 (pandas, NumPy, scikit-learn) with a standard-library HTTP server and a plain HTML, CSS and JavaScript dashboard. The contract is built and tested with Hardhat and ethers. The dashboard has a simulated mode, a live mode that refreshes every 12 s from mainnet, and a viewer mode for the stored database. It shows summary tiles, alerts over time, active cases by severity, the most common signals, a filterable case list, and a case panel organised as *Detect*, *Explain* and *Prove*.

Anchoring uses a file bridge. When an analyst queues a case, the dashboard writes `data/outbox.json`. The script `npm run anchor` reads it, signs with the deployer key, calls `raiseAlert` and, if the analyst had already decided the case, `updateStatus`, and writes the alert ID, transaction hash and gas used to `data/anchored.json`, which the dashboard merges. The script is idempotent: a second run sends no duplicates. `RiskRegistry` is deployed on Sepolia at `0x1D5a2F95...190924b` (block 12053731).

VII. Design Rationale

Why detection is off-chain. Evaluating windows of behaviour for every transfer would need thousands of storage

reads per block, which is far beyond any reasonable gas budget, and Isolation Forest cannot run in the EVM at all. Only the verdict, a few dozen bytes per alert, is worth paying to store permanently.

Why a file bridge for anchoring. A web server that can sign transactions becomes a high-value target, since any flaw in a page or endpoint could then spend from the wallet. Splitting the roles means the server can only write a queue file, and the only code that touches a key is a short script that validates every item and caps the number of transactions per run. The analyst's decision, not a script, decides what reaches the chain.

Why noisy-OR. A weighted sum can reach 100 from many weak signals and is hard to bound. Noisy-OR keeps the score in range, treats signals as independent pieces of evidence, and lets an analyst reproduce the number by hand from the listed reasons. This is the property regulators and auditors need when they ask why a wallet was flagged.

Why a bitmask and hashes only. A 16-bit mask stores the full set of reasons in one storage word alongside the score, which keeps `raiseAlert` cheap, and storing only hashes and public addresses keeps personal data off a ledger that cannot be erased.

Reproducibility. The code, tests and deployment record are in a public repository. The test commands are `npx hardhat test` (15 contract tests), `npm run test:anchor` (8 tests) and `python -m pytest` (54 tests); the dashboard starts with `python -m monitor.serve --demo` or `--live`, and anchoring runs with `npm run anchor`. Results in Section VIII used a fixed seed (7) for the simulator.

VIII. Evaluation

A. Automated tests

The contract has 15 test cases (Table VI); eight are negative tests that expect a revert. All pass. The anchoring logic has 8 further tests against a fake contract, and the Python engine, dashboard API and security checks have 54 tests, all passing. Testing on Windows exposed a real defect: a rejected request whose body was unread reset the connection, so the client saw a dropped connection instead of the error. The server now discards a bounded amount of the unread body before replying, and a regression test covers every rejection path.

B. Detector on simulated fraud

A simulator generated 3,546 transfers, of which 552 belong to 42 injected scenarios, six for each of seven patterns. The alert threshold was 50 and the random seed 7. Table VII gives the totals and Table VIII the per-pattern outcome. Every one of the 42 scenarios was detected and carried the expected reason code. We measure detection per scenario: 248 alerts fell inside the injected scenarios, so not every injected transfer raised its own alert, and we make no per-transfer recall claim.

The 161 extra alerts without the allowlist come from legitimate high-volume wallets, such as exchanges and routers, whose behaviour resembles layering. The allowlist removes them and lifts precision from 0.606 to 0.996. This result must be read with care: the scenarios were written together with the rules, and the allowlist contains the simulator's own exchange addresses. The figures show that the pipeline works as designed, not how accurate it would be against real laundering.

TABLE VI
Smart contract test cases (N = negative, expects revert)

ID	Scenario	Result
TC-01	Deployer is owner and analyst	Pass
TC-02	Analyst anchors alert; event emitted	Pass
TC-03	Look up alert by transaction hash	Pass
TC-04	N: non-analyst raises alert	Pass
TC-05	N: same transaction reported twice	Pass
TC-06	N: invalid score, empty reasons, zero hash or wallet	Pass
TC-07	Open → Reviewed → Escalated emits events	Pass
TC-08	N: change a final state	Pass
TC-09	N: move back to Open or repeat a state	Pass
TC-10	N: outsider updates; unknown alert ID	Pass
TC-11	Owner adds and removes watchlist entry	Pass
TC-12	N: non-owner edits watchlist; empty reason	Pass
TC-13	Revoked analyst loses write access	Pass
TC-14	Two-step ownership transfer	Pass
TC-15	N: non-owner manages analysts; zero address	Pass

TABLE VII
Detection results on simulated data

Metric	With allowlist	Without
Alerts	249	409
Alerts inside fraud scenarios	248	248
False-positive alerts	1	161
False-positive rate (of transfers)	0.03%	5.38%
Alert precision	0.996	0.606
Scenarios detected	42/42	42/42

C. Live mainnet run

In live mode one run ingested 2,365 real transfers and raised 665 alerts. Mainnet has no ground truth, so we claim no accuracy. Many flagged wallets were high-volume services. The dashboard labels wallets that look like exchanges or bots with "possible exchange/bot" and advises checking ownership before escalating, which confirms that real traffic produces false positives an analyst must resolve.

D. On-chain cost

Table IX lists the gas measured on Sepolia. The figures come from single runs on a testnet and were not tuned.

The `updateStatus` figure is derived from the reported fee of 0.00003534 ETH at a gas price of 1 Gwei.

E. Case studies

Alert #0. A wallet scored 82 (critical) with mask 266: high velocity, fan-out and ML outlier. Its evidence showed

TABLE VIII
PER-PATTERN OUTCOME (SIX SCENARIOS EACH)

Pattern	Detected	Expected reason code
Structuring	6/6	6/6
Fan-out	6/6	6/6
Fan-in	6/6	6/6
Round-trip	6/6	6/6
Watchlisted counterparty	6/6	6/6
New-wallet burst	6/6	6/6
High velocity	6/6	6/6

TABLE IX
GAS MEASURED ON SEPOLIA

Operation	Gas used
Contract deployment (block 12053731)	6,426,712
`raiseAlert`, alert #0 (block 12062466)	683,007
`raiseAlert`, alert #1 (block 12066338)	585,087
`updateStatus` (block 12066339)	$\approx$ 35,300

26 transfers by one wallet within 60 minutes split across 26 recipients. The dashboard also labelled it a possible exchange or bot, so the guidance told the analyst to check ownership first. The block explorer shows the `AlertRaised` event with alert ID 0, score 82 and mask 266.

Alert #1. A wallet scored 68 (high) with mask 260, structuring plus ML outlier, from four alerts of repeated transfers just under the 10 ETH limit. The analyst added the note "repeated sub-threshold transfers" and escalated the case. Anchoring produced `AlertRaised` for alert ID 1 (score 68, mask 260), and a second transaction produced `AlertStatusChanged` from Open to Escalated. Escalated is final, so the decision can no longer be altered.

IX. DISCUSSION AND LIMITATIONS

Keeping detection off-chain and only the verdict on-chain limits cost while making the record permanent and role-controlled. The noisy-OR score is easy to explain but its weights are expert-chosen rather than learned. The limitations are:

- Evaluation uses simulated fraud designed alongside the rules, so high scenario recall is expected; real laundering is more varied.
- Thresholds and weights are hand-chosen and uncalibrated, and the mainnet run has no labels.
- Only native ETH transfers are monitored; ERC-20 tokens, internal transfers and cross-chain flows are not.
- Exchanges raise false positives, and the allowlist is a manual file.
- Anchoring depends on one signing key; a multi-signature scheme would be stronger.
- The contract runs on a testnet and has not had a formal audit.

X. CONCLUSION AND FUTURE WORK

We built and tested a monitoring layer that ingests live Ethereum data, scores behaviour with explainable reasons, supports analyst review, and anchors alerts and decisions in a role-based smart contract. All 42 injected scenarios were detected, all 15 contract tests passed, and real alerts were anchored on Sepolia. Live data showed that exchange traffic causes false alarms, which supports the human-in-the-loop design. Future work includes monitoring ERC-20 and internal transfers, adding graph features and labelled data to calibrate thresholds, batching alerts to reduce gas, requiring multi-signature approval for anchoring, deploying on a low-cost layer-2 network, and sending alerts to compliance tools by webhook.

REFERENCES

[1] M. Weber *et al.*, "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," in *KDD Workshop on Anomaly Detection in Finance*, 2019, arXiv:1908.02591.

[2] W. Chen, X. Guo, Z. Chen, Z. Zheng, and Y. Lu, "Phishing scam detection on Ethereum: Towards financial security for blockchain ecosystem," in *Proc. 29th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2020, pp. 4506–4512.

[3] M. U. Hassan, M. H. Rehmani, and J. Chen, "Anomaly detection in blockchain networks: A comprehensive survey," arXiv:2112.06089, 2022.

[4] B. Luo, Z. Zhang, Q. Wang, A. Ke, S. Lu, and B. He, "AI-powered fraud detection in decentralized finance: A project life cycle perspective," arXiv:2308.15992, 2023.

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[6] Financial Action Task Force, "Updated guidance for a risk-based approach to virtual assets and virtual asset service providers," Oct. 2021. [Online]. Available: https://www.fatf-gafi.org

[7] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project Yellow Paper, 2014.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[9] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[10] Ethereum Foundation, "Solidity documentation." [Online]. Available: https://docs.soliditylang.org